

MASTER INTEGRATION PLAN: CLI Agent Features into HelixCode

Document Version: 1.0
Date: 2025-01-20
Status: Draft for Review
Classification: Internal Engineering
Author: Integration Architecture Team

Section 1: EXECUTIVE SUMMARY

1.1 Vision Statement

HelixCode will become the unified, enterprise-grade AI-native code intelligence platform by systematically incorporating the best-of-breed capabilities from 10 leading CLI agent implementations (Claude Code, Aider, Cline, Codex, Plandex, Forge, Kilo Code, OpenCode, Gemini CLI, Amazon Q). This integration transforms HelixCode from a modular architecture into a cohesive “super-agent” capable of autonomous planning, sandboxed execution, multi-model orchestration, and enterprise-scale context management – all while maintaining the Go-based performance, Actor Model concurrency, and PostgreSQL/Redis persistence that define HelixCode’s core identity.

The vision is not to bolt features onto HelixCode, but to **deeply integrate** each capability into the existing architectural patterns: Cobra commands become slash commands, Gin handlers become MCP endpoints, Actor agents become specialized sub-agents, and the Tree-sitter code map becomes the foundation for context compaction and fuzzy matching.

1.2 Scope and Boundaries

IN SCOPE

- **4 missing submodules:** HelixAgent, HelixLLM, HelixMemory, HelixSpecifier (addition via SSH)
- **3 uninitialized submodules:** LLMsVerifier, HelixQA, Challenges, Containers (initialization and wiring)
- **20+ major feature domains** across 10 CLI agent sources (see Feature Inventory below)
- **Build system unification:** Go 1.26 workspace across all submodules
- **Configuration unification:** Single Viper config tree with environment overlays
- **Database schema evolution:** PostgreSQL migrations for tool persistence, sessions, transcripts
- **Redis data structures:** Context windows, caches, session state
- **API surface expansion:** OpenAPI 3.0 spec updates for new endpoints
- **Testing integration:** 100% coverage mandate with HelixQA challenge sessions

OUT OF SCOPE (for this plan)

- **Custom model training or fine-tuning** (inference only)
- **Mobile application development** (CLI and desktop UI only)
- **Third-party marketplace / plugin store** (plugin system architecture only, no store)
- **Billing/subscription engine** (authentication only, billing deferred)
- **Non-code LLM use cases** (creative writing, general chat – code-focused only)

BOUNDARIES

- HelixCode remains the PRIMARY repository; HelixAgent is integrated as a submodule, not as a monorepo parent
- All LLM communication flows through the HelixCode gateway (no submodule direct provider access)
- Sandboxed execution is containerized (Docker/Podman) – no native OS process spawning without sandbox
- PostgreSQL is the single source of truth; Redis is ephemeral/cache only

1.3 Success Criteria

#	Criterion	Measurement	Target
1	All 4 missing submodules added and building	<code>git submodule status + go build ./...</code>	100% clean
2	All 3 uninitialized submodules active	Submodule status + CI pipeline	All initialized
3	Feature parity with Claude Code	Feature checklist (see Appendix A)	95%+ coverage
4	Aider dual-model architecture	Integration test with architect/editor	Passing
5	MCP lifecycle complete	MCP test suite (4 transports + OAuth)	All passing
6	Context compaction operational	1M+ token context handling	<2s compaction
7	Sandboxed shell execution	Security audit + functional test	No escapes
8	Plan/Act mode separation	User acceptance test	Seamless switching
9	100% test coverage	<code>go test -cover</code>	>= 100% (all new code)
10	HelixQA challenge sessions	Challenge completion rate	>90% pass
11	OpenTelemetry observability	Trace coverage	All major flows traced
12	Performance baseline	p95 latency for key operations	<500ms edit, <2s plan
13	Theme system operational	UI rendering test	6 platform themes render
14	Subagent delegation	Multi-agent workflow test	3+ agents cooperate
15	LSP integration	Diagnostics feedback loop	<1s roundtrip

Section 2: PHASES OVERVIEW

Phase 0: Foundation – Submodule Integration & Build System Fix

Attribute	Detail
Phase Goal	Establish the complete repository structure with all 11 submodules (7 existing + 4 missing) properly linked, initialized, and building under a unified Go workspace. Fix SSH submodule resolution and build chain.

Table 2 – continued

Attribute	Detail
Entry Criteria	- Access to all repositories via SSH keys configured- Go 1.26 installed and verified- Current HelixCode builds successfully (baseline)- Submodule access rights confirmed for all 11 repos
Exit Criteria	- <code>git submodule status</code> shows all 11 submodules at expected commits- <code>go work init + go work use</code> produces clean workspace- <code>go build ./...</code> from HelixCode root succeeds- All submodule <code>go.mod</code> modules resolve without replace directives (or with approved ones)- CI pipeline passes on all platforms (Linux, macOS, Windows)
Duration Estimate	5-7 calendar days
Risk Level	HIGH -SSH submodule topology is unusual (HelixCode is child of HelixAgent); circular dependency risk exists
Rollback Strategy	- Pre-phase git tag <code>pre-phase0-baseline</code> on HelixCode- Document original <code>.gitmodules</code> before modification- Keep backup branch <code>backup/pre-submodule-merge</code> - If build fails, revert <code>.gitmodules</code> and <code>go.work</code>

Phase 1: Core Infrastructure – LLM Gateway, Memory, Spec Engine

Attribute	Detail
Phase Goal	Build the three foundational services that all subsequent features depend on: a unified LLM gateway (incorporating OpenCode' s 75+ providers and multi-provider backend), a persistent memory/retrieval system (HelixMemory), and a specification engine (HelixSpecifier) for task decomposition and planning.
Entry Criteria	- Phase 0 exit criteria met- HelixLLM submodule builds independently- HelixMemory PostgreSQL schema designed- HelixSpecifier module interfaces defined

Table 3 – continued

Attribute	Detail
Exit Criteria	- LLM gateway routes to ≥ 5 providers with provider-specific config- Memory system persists and retrieves conversation context with embedding search- Spec engine decomposes a user prompt into actionable subtasks- All three systems have $\geq 90\%$ unit test coverage- OpenAPI spec updated with <code>/api/v1/llm</code> , <code>/api/v1/memory</code> , <code>/api/v1/spec</code> endpoints- Redis connection pools configured for context caching
Duration Estimate	14-18 calendar days
Risk Level	HIGH -LLM provider API drift; embedding model compatibility; spec engine accuracy
Rollback Strategy	- Feature flags for each service (<code>--enable-llm-gateway</code> , <code>--enable-memory</code> , <code>--enable-spec</code>)- Database migrations are reversible (down migrations tested)- Old provider factory remains accessible via <code>LEGACY_PROVIDER_FACTORY=1</code>

Phase 2: CLI Agent Foundation – Tool Framework, Context Management, Edit System

Attribute	Detail
Phase Goal	Implement the core agent primitives: a pluggable tool framework (inspired by Claude Code' s tool system), context management with auto-compaction (Codex + Plandex), and a smart file editing system (Aider' s 4-layer fuzzy matching + Claude Code' s no-flicker rendering).
Entry Criteria	- Phase 1 exit criteria met- Tool interface contracts defined- Context window size limits configured per provider- File watcher / Tree-sitter integration tested

Table 4 – continued

Attribute	Detail
Exit Criteria	- Tool framework supports ≥ 10 tool types with registration/discovery- Context auto-compaction reduces 2M tokens to provider limit with $< 5\%$ semantic loss- Smart file edit applies changes with $\geq 99\%$ accuracy on test corpus- Fuzzy matching resolves file references with $\geq 95\%$ success- No-flicker rendering tested on 6 UI platforms- Session transcript persistence to PostgreSQL
Duration Estimate	18-22 calendar days
Risk Level	MEDIUM-HIGH -Context compaction quality is hard to validate; file edit accuracy depends on Tree-sitter coverage
Rollback Strategy	- <code>AUTO_COMPACTION=off</code> config flag- Legacy edit system preserved in <code>pkg/edit/legacy</code> - Context size can be manually capped

Phase 3: Power Features – Plan Mode, Sandboxing, MCP, Permissions

Attribute	Detail
Phase Goal	Deliver the differentiating agent capabilities: Plan/Act dual mode (Cline, Gemini CLI, Claude Code), sandboxed shell execution (Claude Code, Codex), full MCP lifecycle with OAuth (Claude Code), permission rule system with 5 modes + wildcards (Claude Code), background task system, and hook-based extensibility.
Entry Criteria	- Phase 2 exit criteria met- Container runtime available (Docker/Podman)- MCP spec 2024-11-05 compliance target set- Permission policy schema designed

Table 5 – continued

Attribute	Detail
Exit Criteria	- Plan mode generates actionable plans with user approval checkpoints- Sandboxed shell executes commands with seccomp-bpf + namespace isolation- MCP client connects via all 4 transports (stdio, SSE, HTTP, WebSocket) with OAuth 2.1- Permission system enforces 5 modes (Allow, Deny, Ask, Skip, Review) with wildcard patterns- Background tasks run async with progress reporting- Hook system fires 9+ events with registered handlers- Git worktree isolation tested for concurrent sessions
Duration Estimate	21-28 calendar days
Risk Level	HIGH -Sandboxed execution security is mission-critical; MCP OAuth complexity; permission system must not lock users out
Rollback Strategy	- <code>SANDBOX_MODE=disabled</code> for emergency fallback- Permission default is <code>ask</code> (never deny by default)- MCP can be disabled per-transport- Plan mode has <code>auto-approve</code> opt-in only

Phase 4: UI/UX & TUI – Streaming TUI, Themes, Terminal Intellisense

Attribute	Detail
Phase Goal	Transform the HelixCode user interface into a best-in-class TUI with streaming updates, theme support, terminal intellisense (Amazon Q style), AskUserQuestion with previews (Claude Code), cumulative diff review (Plandex), and no-flicker rendering across all 6 supported platforms.
Entry Criteria	- Phase 3 exit criteria met- UI rendering pipeline benchmarked- Theme asset pipeline defined- Terminal capability detection library chosen

Table 6 – continued

Attribute	Detail
Exit Criteria	- Streaming TUI renders token-by-token with <50ms latency- Theme system applies 6 platform themes + custom themes- Terminal intellisense suggests commands with 85%+ relevance- AskUserQuestion renders file previews, diffs, and images- Cumulative diff review allows accept/reject per hunk- No-flicker rendering verified on: Linux terminal, macOS terminal, Windows Terminal, VS Code terminal, JetBrains terminal, iTerm2
Duration Estimate	14-18 calendar days
Risk Level	MEDIUM –Cross-platform terminal behavior variance; theme asset maintenance
Rollback Strategy	- UI_MODE=simple falls back to basic output- Themes can be disabled with THEME=none- Terminal intellisense is opt-in (--intellisense)

Phase 5: Testing & QA – 100% Coverage, Challenges, HelixQA Sessions

Attribute	Detail
Phase Goal	Achieve production readiness through comprehensive testing: 100% code coverage on all new code, HelixQA automated challenge sessions, LLMsVerifier integration for model output validation, Containers-based integration testing, and performance benchmarking against baselines.
Entry Criteria	- Phase 4 exit criteria met- Test infrastructure can execute containerized tests- Challenge dataset prepared (>= 50 challenges)- Performance benchmarks defined

Table 7 – continued

Attribute	Detail
Exit Criteria	- go test -cover reports $\geq 100\%$ for all new packages- HelixQA completes ≥ 50 challenge sessions with $>90\%$ pass rate- LLMsVerifier validates model outputs with $<5\%$ false positive rate- Integration tests run in Containers with full feature matrix- Performance benchmarks meet all p95 targets- Security audit of sandboxed execution passes- Load test: 100 concurrent sessions stable for 1 hour
Duration Estimate	14-18 calendar days
Risk Level	MEDIUM -100% coverage is time-consuming; challenge flakiness; load test infrastructure
Rollback Strategy	- Coverage gates can be temporarily lowered to 95% with technical debt ticket- Challenges can be rerun; flaky tests are quarantined- Load tests run on staging environment only

Section 3: DETAILED TASKS PER PHASE

Phase 0: Foundation – Submodule Integration & Build System Fix

P0-T1: Audit Current Submodule State

- **Task ID:** P0-T1
- **Task Name:** Complete Submodule Audit and Inventory
- **Description:** Catalog all 87 existing submodules, identify which are initialized, which have SSH URL issues, which have diverged from expected commits. Document the circular dependency between HelixCode (child) and HelixAgent (parent).
- **Dependencies:** None
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** HelixCode (all 87 submodules)
- **Files/Modules to Modify:** SUBMODULE_AUDIT.md (new document)
- **Acceptance Criteria:** Complete inventory table with 87 submodules showing: name, path, current commit, initialized status, SSH URL, last update date, build status.

P0-T2: Add Missing Submodules

- **Task ID:** P0-T2
- **Task Name:** Add HelixAgent, HelixLLM, HelixMemory, HelixSpecifier Submodules

- **Description:** Add the 4 missing submodules as SSH-based git submodules at the correct paths. Handle the special case where HelixAgent is a parent/ancestor of HelixCode by using relative paths or explicit SSH URLs.
- **Dependencies:** P0-T1
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** HelixAgent, HelixLLM, HelixMemory, HelixSpecifier
- **Files/Modules to Modify:** .gitmodules, .git/config (local)
- **Acceptance Criteria:** `git submodule status` shows all 4 new submodules; `git submodule update --init --recursive` clones all 4 successfully; no SSH key prompts during CI.

P0-T3: Initialize Dormant Submodules

- **Task ID:** P0-T3
- **Task Name:** Initialize LLMsVerifier, HelixQA, Challenges, Containers
- **Description:** The 4 present-but-uninitialized submodules need to be initialized and their builds verified. Check if they have independent build requirements (e.g., Containers may need Docker, HelixQA may need test datasets).
- **Dependencies:** P0-T2
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** LLMsVerifier, HelixQA, Challenges, Containers
- **Files/Modules to Modify:** .gitmodules (update init paths if needed), submodule README.md files
- **Acceptance Criteria:** All 4 submodules show `init` status; each submodule's `go build ./...` (or equivalent) succeeds.

P0-T4: Create Go Workspace

- **Task ID:** P0-T4
- **Task Name:** Unify Build System with Go Workspaces
- **Description:** Create a `go.work` file at HelixCode root that includes all submodules with Go code. Resolve any module path conflicts, replace directives, or version mismatches. Ensure the workspace builds from a clean environment.
- **Dependencies:** P0-T2, P0-T3
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** All Go-based submodules
- **Files/Modules to Modify:** `go.work` (new), `go.work.sum` (generated), individual `go.mod` files if version alignment needed
- **Acceptance Criteria:** `go work sync` completes without errors; `go build ./...` from HelixCode root builds all modules; `go test ./...` runs all tests.

P0-T5: CI Pipeline for Submodule Matrix

- **Task ID:** P0-T5
- **Task Name:** GitHub Actions CI for Submodule Build Matrix
- **Description:** Create or update GitHub Actions workflows to build and test across all submodules on Linux, macOS, and Windows. Include SSH key setup for private submodule access.
- **Dependencies:** P0-T4
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** All (CI configuration)

- **Files/Modules to Modify:** `.github/workflows/ci.yml` (new or updated), `.github/workflows/submodule-check.yml`
 - **Acceptance Criteria:** CI passes for all 11 submodules on all 3 OS platforms; submodule drift detection job runs daily.
-

Phase 1: Core Infrastructure – LLM Gateway, Memory, Spec Engine

P1-T1: LLM Gateway Interface Design

- **Task ID:** P1-T1
- **Task Name:** Define Unified LLM Provider Interface
- **Description:** Design the `LLMGateway` interface that abstracts all 29+ existing providers and prepares for 75+ (OpenCode-style). Must support: completion, streaming, tool calling, multi-modal input, context window querying, and provider capability discovery.
- **Dependencies:** P0-T5 (workspace ready)
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** `HelixLLM`, `HelixCode` (internal LLM factory)
- **Files/Modules to Modify:** `pkg/llm/gateway.go` (new), `pkg/llm/provider.go` (new), `pkg/llm/capability.go` (new)
- **Acceptance Criteria:** Interface compiles; mock implementation passes; 29 existing providers can be adapted to interface.

P1-T2: Multi-Provider Backend Integration

- **Task ID:** P1-T2
- **Task Name:** Integrate 75+ Provider Support with Mid-Session Switching
- **Description:** Port OpenCode's provider registry to `HelixCode`. Implement dynamic provider loading, configuration per provider (API keys, base URLs, timeouts), and mid-session model switching. Add provider health checking.
- **Dependencies:** P1-T1
- **Estimated Effort:** 4 days
- **Submodule(s) Involved:** `HelixLLM`, `OpenCode` (reference)
- **Files/Modules to Modify:** `pkg/llm/registry.go`, `pkg/llm/provider_*.go` (one per provider family), `pkg/llm/switch.go`, `config/providers.yaml`
- **Acceptance Criteria:** Can list 75+ providers; can switch provider mid-conversation without losing context; health check returns status for each provider.

P1-T3: Provider Wizard and Configuration

- **Task ID:** P1-T3
- **Task Name:** Multi-Provider Backend Wizards
- **Description:** Interactive CLI wizards (Cobra-based) for configuring new providers. Prompt for API keys, test connectivity, save to Viper config. Support OAuth flows for providers requiring it.
- **Dependencies:** P1-T2
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** `HelixCode` (CLI)
- **Files/Modules to Modify:** `cmd/llm/wizard.go`, `cmd/llm/configure.go`, `pkg/config/provider.go`

- **Acceptance Criteria:** `helixcode llm configure --provider openai` completes interactively; config persists; connection test passes.

P1-T4: Memory System Schema and Storage

- **Task ID:** P1-T4
- **Task Name:** PostgreSQL Schema for Conversation Memory
- **Description:** Design and implement PostgreSQL schema for persistent memory: conversations, messages, embeddings, session state, and context windows. Use pgvector for embedding storage.
- **Dependencies:** P0-T5
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixMemory
- **Files/Modules to Modify:** `migrations/001_memory.up.sql`, `migrations/001_memory.down.sql`, `pkg/memory/schema.go`, `pkg/memory/store.go`
- **Acceptance Criteria:** Migrations apply cleanly; pgvector extension enabled; CRUD operations on all tables pass.

P1-T5: Embedding and Retrieval Engine

- **Task ID:** P1-T5
- **Task Name:** Semantic Search with Embeddings
- **Description:** Integrate embedding generation (via HelixLLM or local model) and semantic retrieval. Implement conversation context retrieval, file similarity search, and cross-session memory. Support HNSW indexing.
- **Dependencies:** P1-T4, P1-T2
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixMemory, HelixLLM
- **Files/Modules to Modify:** `pkg/memory/embed.go`, `pkg/memory/search.go`, `pkg/memory/index.go`
- **Acceptance Criteria:** Can store 1000 conversation fragments and retrieve top-5 relevant in <100ms; embedding dimension matches model output.

P1-T6: Spec Engine Core

- **Task ID:** P1-T6
- **Task Name:** Task Decomposition and Specification Engine
- **Description:** Build HelixSpecifier core: parse user intent, decompose into subtasks, generate specifications for each subtask. Uses LLM with structured output (JSON schema) for reliable decomposition.
- **Dependencies:** P1-T2
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixSpecifier
- **Files/Modules to Modify:** `pkg/spec/engine.go`, `pkg/spec/decomposer.go`, `pkg/spec/validator.go`, `pkg/spec/schema.go`
- **Acceptance Criteria:** “Implement a REST API with auth” decomposes into ≥ 3 subtasks with clear inputs/outputs; decomposition succeeds 90% of time on benchmark prompts.

P1-T7: Redis Context Cache

- **Task ID:** P1-T7

- **Task Name:** Redis-Based Context Window Caching
- **Description:** Implement Redis data structures for caching LLM context windows, provider responses, and embedding vectors. Configure TTL policies and eviction strategies.
- **Dependencies:** P1-T4
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** HelixCode (Redis layer)
- **Files/Modules to Modify:** pkg/cache/redis.go, pkg/cache/context.go, pkg/cache/ttl.go
- **Acceptance Criteria:** Context cache hit reduces LLM call latency by $\geq 50\%$; TTL eviction works; Redis memory usage bounded.

P1-T8: OpenAPI Spec Update for Core APIs

- **Task ID:** P1-T8
- **Task Name:** Expand OpenAPI 3.0 Spec for New Endpoints
- **Description:** Add /api/v1/llm/*, /api/v1/memory/*, /api/v1/spec/* endpoints to the OpenAPI specification. Generate handler stubs and client SDK updates.
- **Dependencies:** P1-T1, P1-T4, P1-T6
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode (Gin server)
- **Files/Modules to Modify:** api/openapi.yaml, pkg/api/handlers_llm.go, pkg/api/handlers_memory.go, pkg/api/handlers_spec.go
- **Acceptance Criteria:** openapi-generator produces valid client; all new endpoints documented; handler stubs compile.

Phase 2: CLI Agent Foundation – Tool Framework, Context Management, Edit System

P2-T1: Tool Framework Core

- **Task ID:** P2-T1
- **Task Name:** Pluggable Tool Framework with Registration/Discovery
- **Description:** Implement a tool framework where each tool is a self-describing plugin with JSON schema for inputs/outputs. Tools register at startup, support validation, execution, and result formatting. Inspired by Claude Code's tool system.
- **Dependencies:** P1-T2 (LLM gateway ready)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode (core), HelixAgent (reference)
- **Files/Modules to Modify:** pkg/tool/framework.go, pkg/tool/registry.go, pkg/tool/executor.go, pkg/tool/schema.go, pkg/tool/builtin/*.go
- **Acceptance Criteria:** ≥ 10 tools registered; tool discovery API lists all tools; tool execution returns structured results; schema validation rejects invalid inputs.

P2-T2: Built-in Tool Suite

- **Task ID:** P2-T2
- **Task Name:** Implement Core Tool Suite (Read, Edit, Bash, Git, Search, LS, Glob, Ask, Done)
- **Description:** Build the essential tools every code agent needs: file read/write, directory listing, glob search, grep search, bash execution, git operations, user ask, and task completion signaling.
- **Dependencies:** P2-T1

- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tool/read.go, pkg/tool/edit.go, pkg/tool/bash.go, pkg/tool/git.go, pkg/tool/search.go, pkg/tool/ls.go, pkg/tool/glob.go, pkg/tool/ask.go, pkg/tool/done.go
- **Acceptance Criteria:** Each tool has unit tests; integration test exercises all tools in sequence; tool results are serializable to JSON.

P2-T3: Tool Result Persistence

- **Task ID:** P2-T3
- **Task Name:** Persist Tool Results to PostgreSQL
- **Description:** Store all tool invocations, inputs, outputs, and execution timestamps in PostgreSQL. Support querying tool history by session, by tool type, and by time range.
- **Dependencies:** P2-T2, P1-T4
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixMemory
- **Files/Modules to Modify:** pkg/tool/persistence.go, migrations/002_tool_results.up.sql, pkg/tool/history.go
- **Acceptance Criteria:** Every tool call stored; query by session returns chronological list; no duplicate storage of identical calls.

P2-T4: Context Window Manager

- **Task ID:** P2-T4
- **Task Name:** Context Window Tracking and Limits
- **Description:** Track token usage per provider, per model, per session. Enforce context window limits and provide warnings at 80%, 90%, 95% thresholds.
- **Dependencies:** P1-T2, P1-T7
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/context/window.go, pkg/context/tokenizer.go, pkg/context/limits.go
- **Acceptance Criteria:** Accurate token counting (within 5% of provider count); warnings fire at correct thresholds; hard limit prevents overflow.

P2-T5: Auto-Compaction System

- **Task ID:** P2-T5
- **Task Name:** Semantic Context Compaction (Codex + Plandex Style)
- **Description:** Implement automatic context compaction when approaching token limits. Strategy: summarize older conversation turns, compress file contents with Tree-sitter structural summaries, and evict least-relevant context based on embedding similarity.
- **Dependencies:** P2-T4, P1-T5, P1-T7
- **Estimated Effort:** 4 days
- **Submodule(s) Involved:** HelixCode, HelixLLM, HelixMemory
- **Files/Modules to Modify:** pkg/context/compactor.go, pkg/context/summarizer.go, pkg/context/relevance.go

- **Acceptance Criteria:** 2M token context compacted to 128k with <5% semantic loss (measured by QA task accuracy); compaction time <2s.

P2-T6: Smart File Editing with Fuzzy Matching

- **Task ID:** P2-T6
- **Task Name:** 4-Layer Fuzzy Matching Edit System (Aider-Inspired)
- **Description:** Implement Aider's 4-layer approach: (1) exact match, (2) normalized whitespace match, (3) indent-ignoring match, (4) AI-guided fuzzy match. Apply edits with confidence scoring and fallback to user confirmation.
- **Dependencies:** P2-T2 (edit tool), P1-T6 (spec engine for guidance)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode, HelixAgent
- **Files/Modules to Modify:** pkg/edit/matcher.go (4 layers), pkg/edit/applier.go, pkg/edit/confidence.go
- **Acceptance Criteria:** 99%+ accuracy on exact match test corpus; 95%+ on fuzzy match corpus; <0.1% data loss incidents.

P2-T7: No-Flicker Rendering Engine

- **Task ID:** P2-T7
- **Task Name:** Terminal Rendering with No Flicker (Claude Code Style)
- **Description:** Implement buffered terminal output that computes minimal diff between frames and renders only changed regions. Use terminal capability detection (CSI queries) for feature selection.
- **Dependencies:** P2-T6
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode (UI layer)
- **Files/Modules to Modify:** pkg/render/buffer.go, pkg/render/diff.go, pkg/render/terminal.go, pkg/render/capabilities.go
- **Acceptance Criteria:** 60 FPS render of streaming text with zero visible flicker on tested terminals; falls back gracefully on dumb terminals.

P2-T8: Session Transcript and Resume

- **Task ID:** P2-T8
- **Task Name:** Session Transcript Persistence and Resume (Claude Code Style)
- **Description:** Persist full session transcripts (user inputs, AI responses, tool calls, results) to PostgreSQL. Support resuming a session from any point with full context restoration.
- **Dependencies:** P2-T3, P1-T4
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixMemory
- **Files/Modules to Modify:** pkg/session/transcript.go, pkg/session/resume.go, pkg/session/serialize.go
- **Acceptance Criteria:** Session saved after every turn; resume restores identical context; 100 sessions can be listed and selected.

P2-T9: Background Task System

- **Task ID:** P2-T9

- **Task Name:** Async Background Task Execution
 - **Description:** Allow tasks (tool calls, file operations, LLM requests) to run in background with progress reporting. Support task cancellation, prioritization, and result notification.
 - **Dependencies:** P2-T1, P1-T7
 - **Estimated Effort:** 2 days
 - **Submodule(s) Involved:** HelixCode
 - **Files/Modules to Modify:** pkg/task/manager.go, pkg/task/worker.go, pkg/task/progress.go, pkg/task/cancel.go
 - **Acceptance Criteria:** Task submitted and runs async; progress updates stream to UI; cancellation stops task; results delivered when complete.
-

Phase 3: Power Features – Plan Mode, Sandboxing, MCP, Permissions

P3-T1: Plan Mode Engine

- **Task ID:** P3-T1
- **Task Name:** Plan/Act Dual Mode with Different Models (Cline + Gemini CLI Style)
- **Description:** Implement Plan mode where a planning model (e.g., o1, Claude 3.5 Sonnet) generates a step-by-step plan, and an acting model (e.g., Claude 3.5 Haiku, GPT-4o-mini) executes each step. Plan must be user-editable and require approval before execution.
- **Dependencies:** P1-T6 (spec engine), P1-T2 (multi-provider), P2-T9 (background tasks)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode, HelixSpecifier
- **Files/Modules to Modify:** pkg/plan/engine.go, pkg/plan/approval.go, pkg/plan/executor.go, pkg/plan/state.go
- **Acceptance Criteria:** Plan generated with ≥ 3 steps for complex requests; user can edit plan; approval gates prevent auto-execution; different models selectable for plan vs act.

P3-T2: Cumulative Diff Review Sandbox

- **Task ID:** P3-T2
- **Task Name:** Diff Review Sandbox with Per-Hunk Approval (Plandex Style)
- **Description:** Collect all file changes across a session into a cumulative diff. Present hunks for review with accept/reject/edit options. Maintain a sandbox branch until changes are approved.
- **Dependencies:** P2-T6 (smart edit), P3-T1 (plan mode)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/diff/sandbox.go, pkg/diff/review.go, pkg/diff/hunk.go, pkg/diff/approval.go
- **Acceptance Criteria:** Changes accumulate across session; diff renders with syntax highlighting; accept/reject per hunk works; rejected hunks excluded from final patch.

P3-T3: Shadow Git Checkpoints

- **Task ID:** P3-T3
- **Task Name:** Automatic Git Checkpoints Before Every Change (Cline Style)
- **Description:** Create lightweight git checkpoints (branches or stashes) before every file modification. Support checkpoint restore, diff between checkpoints, and checkpoint history navigation.

- **Dependencies:** P2-T2 (git tool)
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/git/checkpoint.go, pkg/git/restore.go, pkg/git/history.go
- **Acceptance Criteria:** Checkpoint created before each edit; restore returns to exact prior state; checkpoint list navigable; no impact on user's main branch.

P3-T4: Git Worktree Isolation

- **Task ID:** P3-T4
- **Task Name:** Git Worktree Isolation for Concurrent Sessions (Claude Code Style)
- **Description:** Use git worktrees to isolate concurrent agent sessions. Each session gets its own worktree with independent file state, preventing cross-session interference.
- **Dependencies:** P3-T3
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/git/worktree.go, pkg/session/isolation.go
- **Acceptance Criteria:** 5 concurrent sessions each in own worktree; changes in one don't affect others; worktree cleanup on session end.

P3-T5: Sandboxed Shell Execution Core

- **Task ID:** P3-T5
- **Task Name:** Container-Based Sandboxed Shell (Claude Code + Codex Style)
- **Description:** Implement containerized command execution using Docker/Podman. Each shell command runs in a fresh container with limited filesystem access, network restrictions, and timeout enforcement.
- **Dependencies:** P0-T5 (containers submodule), P2-T2 (bash tool)
- **Estimated Effort:** 4 days
- **Submodule(s) Involved:** Containers, HelixCode
- **Files/Modules to Modify:** pkg/sandbox/container.go, pkg/sandbox/executor.go, pkg/sandbox/policy.go, pkg/sandbox/network.go, pkg/sandbox/fs.go
- **Acceptance Criteria:** `ls -la` runs in container; network-restricted container cannot curl; timeout kills after limit; filesystem only sees mounted project directory.

P3-T6: OS-Native Sandboxed Execution

- **Task ID:** P3-T6
- **Task Name:** seccomp-bpf + Namespace Sandboxing for Native Performance (Codex Style)
- **Description:** For local execution without Docker, implement Linux-native sandboxing using seccomp-bpf system call filtering, PID namespaces, mount namespaces, and capability dropping. macOS uses seatbelt/entitlements equivalent.
- **Dependencies:** P3-T5
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/sandbox/seccomp.go, pkg/sandbox/namespace.go, pkg/sandbox/native.go
- **Acceptance Criteria:** `execve` blocked by seccomp; process cannot access files outside project; `setuid` calls fail; sandbox overhead <50ms per command.

P3-T7: Permission Rule System

- **Task ID:** P3-T7
- **Task Name:** 5-Mode Permission System with Wildcards (Claude Code Style)
- **Description:** Implement permission rules with 5 modes: **Allow** (auto-execute), **Deny** (block), **Ask** (prompt user), **Skip** (silently ignore), **Review** (queue for batch review). Support wildcard patterns for file paths, command patterns, and tool types. Configurable per-directory, per-session, or globally.
- **Dependencies:** P2-T1 (tool framework), P3-T5 (sandbox)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/permission/engine.go, pkg/permission/rule.go, pkg/permission/matcher.go, pkg/permission/mode.go, config/permissions.yaml
- **Acceptance Criteria:** All 5 modes tested; wildcard *.log matches correctly; rule precedence resolved (most specific wins); permission prompt rendered in TUI.

P3-T8: MCP Client Lifecycle

- **Task ID:** P3-T8
- **Task Name:** MCP Client with 4 Transports + OAuth (Claude Code Style)
- **Description:** Implement Model Context Protocol client supporting 4 transports: stdio (local process), SSE (server-sent events), HTTP (direct REST), WebSocket (bidirectional). Include full OAuth 2.1 PKCE flow for authentication.
- **Dependencies:** P1-T2 (provider system), P2-T1 (tool framework)
- **Estimated Effort:** 5 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/mcp/client.go, pkg/mcp/transport_stdio.go, pkg/mcp/transport_sse.go, pkg/mcp/transport_http.go, pkg/mcp/transport_ws.go, pkg/mcp/oauth.go, pkg/mcp/discovery.go
- **Acceptance Criteria:** Connects to MCP server via each transport; OAuth flow completes; tool discovery lists remote tools; remote tool execution returns results.

P3-T9: Hook-Based Extensibility

- **Task ID:** P3-T9
- **Task Name:** 9+ Event Hook System (Claude Code Style)
- **Description:** Implement a hook system firing events at key lifecycle points: **before_request**, **after_request**, **before_tool_call**, **after_tool_call**, **before_edit**, **after_edit**, **on_plan**, **on_approval**, **on_error**, **on_session_start**, **on_session_end**. Support pre- and post-hooks with cancellation.
- **Dependencies:** P2-T1, P3-T7
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/hook/engine.go, pkg/hook/events.go, pkg/hook/handler.go
- **Acceptance Criteria:** All 11 hooks fire at correct times; pre-hook can cancel operation; post-hook receives full context; handler registration is dynamic.

P3-T10: Skill System

- **Task ID:** P3-T10
- **Task Name:** Skill Definition and Loading System (Claude Code Style)

- **Description:** Skills are YAML/JSON bundles defining: prompt templates, tool configurations, permission rules, model preferences, and hooks. Load skills from filesystem or remote URL. Support skill versioning and dependencies.
- **Dependencies:** P3-T9, P3-T7
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/skill/loader.go, pkg/skill/schema.go, pkg/skill/manager.go, pkg/skill/version.go
- **Acceptance Criteria:** Skill loaded from YAML; tools configured by skill; permissions overridden by skill; skill dependency resolved.

P3-T11: Subagent Team System

- **Task ID:** P3-T11
- **Task Name:** Subagent Team Delegation (Claude Code + Kilo Code Style)
- **Description:** Create specialized subagent types: Researcher (finds information), Editor (makes file changes), Reviewer (checks quality), Tester (runs tests), Architect (designs structure). Orchestrate via a meta-agent that delegates tasks.
- **Dependencies:** P3-T1, P1-T6, P2-T9
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/agent/roles.go, pkg/agent/orchestrator.go, pkg/agent/delegator.go, pkg/agent/communicator.go
- **Acceptance Criteria:** 5 subagent types defined; meta-agent delegates to correct specialist; inter-agent communication works; results aggregated.

P3-T12: Computer Use / Browser Automation

- **Task ID:** P3-T12
- **Task Name:** Browser Automation for Computer Use (Cline Style)
- **Description:** Integrate browser automation (Playwright/CDP) for computer use capabilities: navigate URLs, click elements, fill forms, take screenshots, and extract content. Return results as tool outputs.
- **Dependencies:** P2-T1 (tool framework)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tool/browser.go, pkg/browser/controller.go, pkg/browser/screenshot.go
- **Acceptance Criteria:** Can navigate to URL and click element; screenshot returned as base64; form fill works; headless mode default.

P3-T13: Quality Scoring and Automated Gates

- **Task ID:** P3-T13
- **Task Name:** Quality Scoring with Automated Pass/Fail Gates (Forge Style)
- **Description:** Implement quality metrics for agent outputs: code style, test coverage, static analysis score, security scan, performance benchmark. Gate plan execution on quality thresholds.
- **Dependencies:** P3-T1, P2-T6
- **Estimated Effort:** 2 days

- **Submodule(s) Involved:** HelixCode, HelixQA
- **Files/Modules to Modify:** pkg/quality/scorer.go, pkg/quality/gate.go, pkg/quality/metrics.go
- **Acceptance Criteria:** Score computed for code output; gate blocks low-quality changes; metric definitions extensible.

P3-T14: OpenTelemetry Integration

- **Task ID:** P3-T14
 - **Task Name:** Distributed Tracing and Metrics (Claude Code Style)
 - **Description:** Integrate OpenTelemetry for tracing all major flows: LLM requests, tool calls, file edits, plan steps, MCP operations. Export to Jaeger/Zipkin and Prometheus.
 - **Dependencies:** P1-T2, P3-T8
 - **Estimated Effort:** 2 days
 - **Submodule(s) Involved:** HelixCode
 - **Files/Modules to Modify:** pkg/telemetry/tracer.go, pkg/telemetry/metrics.go, pkg/telemetry/provider.go
 - **Acceptance Criteria:** Trace spans cover all major operations; p95 latency measured; metrics exported; sampling configurable.
-

Phase 4: UI/UX & TUI – Streaming TUI, Themes, Terminal Intellisense

P4-T1: Streaming TUI Core

- **Task ID:** P4-T1
- **Task Name:** Token-Streaming TUI with Markdown Rendering
- **Description:** Build a TUI that streams LLM output token-by-token, renders markdown (code blocks, lists, tables), and handles concurrent tool output streams. Support split-pane view (conversation + tool output).
- **Dependencies:** P2-T7 (no-flicker render), P3-T5 (sandbox output)
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/stream.go, pkg/tui/markdown.go, pkg/tui/layout.go, pkg/tui/pane.go
- **Acceptance Criteria:** Tokens render within 50ms of arrival; markdown code blocks syntax highlighted; split pane resizable; 60 FPS maintained.

P4-T2: Theme System

- **Task ID:** P4-T2
- **Task Name:** 6-Platform Theme System with Custom Themes (Claude Code Style)
- **Description:** Implement a theme engine supporting: Linux terminal (16/256/truecolor), macOS Terminal.app, Windows Terminal, VS Code integrated terminal, JetBrains terminal, iTerm2. Detect terminal capabilities and adapt. Support custom theme YAML files.
- **Dependencies:** P2-T7 (terminal capabilities)
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode

- **Files/Modules to Modify:** pkg/theme/engine.go, pkg/theme/palette.go, pkg/theme/platforms.go, pkg/theme/custom.go, themes/*.yaml
- **Acceptance Criteria:** Each platform renders with correct colors; custom theme loads and applies; theme switch without restart.

P4-T3: AskUserQuestion with Previews

- **Task ID:** P4-T3
- **Task Name:** Interactive User Prompts with File/Diff/Image Previews (Claude Code Style)
- **Description:** Enhanced AskUserQuestion tool that renders rich previews: file contents with syntax highlighting, diffs with color coding, images in supported terminals (iTerm2 inline, sixel, kitty graphics). Support keyboard navigation and quick actions.
- **Dependencies:** P4-T1, P4-T2, P2-T2 (ask tool)
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/ask.go, pkg/tui/preview_file.go, pkg/tui/preview_diff.go, pkg/tui/preview_image.go
- **Acceptance Criteria:** File preview renders with syntax highlight; diff preview shows colors; image preview works in iTerm2/kitty; keyboard navigation intuitive.

P4-T4: Terminal Intellisense

- **Task ID:** P4-T4
- **Task Name:** Fig-Style Terminal Intellisense (Amazon Q Style)
- **Description:** Provide command-line completion suggestions as you type, showing: available commands, file paths, git branches, recent commands, and contextual suggestions based on current directory contents. Render in a floating popup.
- **Dependencies:** P4-T1
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/intellisense.go, pkg/tui/completion.go, pkg/tui/popup.go, pkg/tui/context.go
- **Acceptance Criteria:** Suggestions appear within 100ms of typing; file path completion works; git branch completion works; popup renders correctly; 85%+ relevance score.

P4-T5: Slash Command System

- **Task ID:** P4-T5
- **Task Name:** Slash Command Registry and Execution (Claude Code Style)
- **Description:** Implement /command system where commands like /compact, /memory, /undo, /help, /model are registered and executed. Commands have completions, help text, and argument parsing.
- **Dependencies:** P4-T4, P2-T1 (tool framework)
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/slash/registry.go, pkg/slash/executor.go, pkg/slash/builtins.go, pkg/slash/completion.go
- **Acceptance Criteria:** >= 15 slash commands registered; tab completion works; help text displayed; arguments parsed correctly.

P4-T6: Architect/Editor Dual-Model UI

- **Task ID:** P4-T6
- **Task Name:** Dual-Model UI for Architect/Editor Mode (Aider Style)
- **Description:** UI mode showing both architect model reasoning and editor model output side by side. Architect view shows design rationale; editor view shows implementation. Support switching focus between views.
- **Dependencies:** P4-T1, P3-T1
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/dual_model.go, pkg/tui/architect_pane.go, pkg/tui/editor_pane.go
- **Acceptance Criteria:** Architect reasoning visible; editor code visible; dual pane layout correct; focus switch works.

P4-T7: Multi-Agent Platform State Machine UI

- **Task ID:** P4-T7
- **Task Name:** Agent State Machine Visualization (Kilo Code Style)
- **Description:** Visualize the multi-agent state machine: show current state (Idle, Planning, Executing, Reviewing, Error), agent assignments, and transitions. Support manual state override.
- **Dependencies:** P3-T11, P4-T1
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/state_machine.go, pkg/tui/agent_status.go, pkg/tui/transition.go
- **Acceptance Criteria:** Current state clearly shown; agent assignments listed; state transitions animated; manual override works.

P4-T8: LSP Integration with Diagnostics

- **Task ID:** P4-T8
- **Task Name:** LSP Client for Diagnostics Feedback (OpenCode Style)
- **Description:** Integrate LSP client for receiving real-time diagnostics from language servers. Show errors, warnings, and suggestions inline. Support multiple concurrent LSP servers.
- **Dependencies:** P4-T1, P2-T6
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/lsp/client.go, pkg/lsp/diagnostics.go, pkg/lsp/server_manager.go, pkg/lsp/inline.go
- **Acceptance Criteria:** LSP connection established; diagnostics displayed inline; multi-server support; <1s roundtrip.

P4-T9: Multimodal Input Processing

- **Task ID:** P4-T9
- **Task Name:** Image and File Attachment in Prompts (Gemini CLI Style)
- **Description:** Support multimodal input: drag-and-drop image files, paste images from clipboard, attach PDFs or text files to prompts. Process with vision-capable models.

- **Dependencies:** P4-T1, P1-T2
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/input/multimodal.go, pkg/input/image.go, pkg/input/clipboard.go, pkg/input/attachment.go
- **Acceptance Criteria:** Image attached and sent to vision model; PDF text extracted; clipboard image pasted; attachment size limits enforced.

P4-T10: 1M+ Token Context Window UI

- **Task ID:** P4-T10
- **Task Name:** Large Context Window Indicator and Management (Gemini CLI Style)
- **Description:** UI components for managing 1M+ token contexts: context usage bar, file inclusion manager, context window visualization, and token budget allocation per file.
- **Dependencies:** P2-T4, P2-T5
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** pkg/tui/context_bar.go, pkg/tui/context_manager.go, pkg/tui/token_budget.go
- **Acceptance Criteria:** Context bar accurate; file inclusion toggle works; budget allocation visible; 1M tokens handled without UI lag.

Phase 5: Testing & QA – 100% Coverage, Challenges, HelixQA Sessions

P5-T1: Unit Test Coverage Enforcement

- **Task ID:** P5-T1
- **Task Name:** Achieve 100% Unit Test Coverage for All New Code
- **Description:** Write unit tests for every new function, method, and branch in Phases 1-4. Use table-driven tests, property-based tests where appropriate, and mock interfaces for external dependencies. Enforce via CI gate.
- **Dependencies:** All Phase 1-4 tasks
- **Estimated Effort:** 5 days
- **Submodule(s) Involved:** All
- **Files/Modules to Modify:** *_test.go files across all new packages
- **Acceptance Criteria:** go test -cover reports >= 100% for all packages modified in Phases 1-4; no test flakes in 100 consecutive runs.

P5-T2: Integration Test Suite

- **Task ID:** P5-T2
- **Task Name:** End-to-End Integration Tests in Containers
- **Description:** Build containerized integration tests exercising full workflows: plan -> edit -> test -> commit. Use Docker Compose for PostgreSQL + Redis + HelixCode stack. Test all major feature combinations.
- **Dependencies:** P5-T1, P0-T5
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** Containers, HelixCode

- **Files/Modules to Modify:** tests/integration/*.go, docker-compose.test.yml, .github/workflows/integration.yml
- **Acceptance Criteria:** ≥ 20 integration scenarios pass; test stack starts in $<30s$; tests complete in <10 minutes.

P5-T3: HelixQA Challenge Sessions

- **Task ID:** P5-T3
- **Task Name:** Automated Challenge-Based QA Testing
- **Description:** Configure HelixQA submodule to run ≥ 50 challenge sessions against the integrated HelixCode. Challenges cover: bug fixing, feature implementation, refactoring, test writing, and documentation. Track pass rate and regression rate.
- **Dependencies:** P0-T3 (HelixQA initialized), P5-T2
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** HelixQA, Challenges
- **Files/Modules to Modify:** helixqa/challenges/*.yaml, helixqa/runner.go, .github/workflows/helixqa.yml
- **Acceptance Criteria:** ≥ 50 challenges defined; $>90\%$ pass rate; challenge results stored; regression detection automated.

P5-T4: LLMsVerifier Integration

- **Task ID:** P5-T4
- **Task Name:** Model Output Verification and Consistency Checks
- **Description:** Integrate LLMsVerifier to validate LLM outputs: check for hallucinations, verify code correctness, detect harmful content, and measure output consistency across providers. Use for both regression testing and runtime validation.
- **Dependencies:** P0-T3 (LLMsVerifier initialized), P1-T2
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** LLMsVerifier, HelixCode
- **Files/Modules to Modify:** pkg/verify/llmverifier.go, pkg/verify/hallucination.go, pkg/verify/consistency.go
- **Acceptance Criteria:** Hallucination detection $<5\%$ false positive; consistency score computed; verifier runs in CI; runtime validation optional.

P5-T5: Security Audit – Sandboxed Execution

- **Task ID:** P5-T5
- **Task Name:** Penetration Testing for Sandbox Escape
- **Description:** Hire/engage security team to attempt sandbox escape from containerized and seccomp-based sandboxes. Test vectors: privilege escalation, filesystem breakout, network tunneling, resource exhaustion, and side-channel attacks.
- **Dependencies:** P3-T5, P3-T6
- **Estimated Effort:** 3 days
- **Submodule(s) Involved:** Containers, HelixCode
- **Files/Modules to Modify:** security/audit/sandbox_escape_tests.go (new)
- **Acceptance Criteria:** No sandbox escapes found; all test vectors contained; security report generated and signed off.

P5-T6: Performance Benchmarking

- **Task ID:** P5-T6
- **Task Name:** Establish and Validate Performance Baselines
- **Description:** Create benchmark suite measuring: p95 edit latency, p95 plan generation time, p95 context compaction time, throughput (requests/sec), memory usage under load. Run against baselines and fail on regression >10%.
- **Dependencies:** All Phase 1-4 tasks
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** tests/benchmark/*.go, .github/workflows/benchmark.yml
- **Acceptance Criteria:** All p95 targets met; benchmarks run in CI; regression alerts configured; results dashboard accessible.

P5-T7: Load and Stress Testing

- **Task ID:** P5-T7
- **Task Name:** 100 Concurrent Session Load Test
- **Description:** Run 100 concurrent agent sessions for 1 hour against the full stack. Monitor: response latency, error rate, memory leaks, goroutine leaks, database connection pool exhaustion, Redis saturation.
- **Dependencies:** P5-T6, P5-T2
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** HelixCode, Containers
- **Files/Modules to Modify:** tests/load/*.go, tests/load/scenarios.yaml
- **Acceptance Criteria:** 100 sessions stable for 1 hour; <1% error rate; no memory leaks detected; no goroutine growth; database and Redis healthy.

P5-T8: Plugin System Validation

- **Task ID:** P5-T8
- **Task Name:** Validate 25+ Lifecycle Hooks for Plugin System (OpenCode Style)
- **Description:** Test the plugin system with 25+ lifecycle hooks: plugin load, init, pre/post command, pre/post tool, pre/post edit, on error, on shutdown. Validate with sample plugins.
- **Dependencies:** P3-T9 (hook system), P4-T5 (slash commands)
- **Estimated Effort:** 1 day
- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** tests/plugin/*.go, tests/plugin/sample_plugin.go
- **Acceptance Criteria:** All 25+ hooks tested; sample plugin loads and runs; plugin isolation verified; plugin crash doesn't crash host.

P5-T9: A/B Testing Framework Validation

- **Task ID:** P5-T9
- **Task Name:** A/B Testing for Agent Configurations (Forge Style)
- **Description:** Validate the A/B testing framework: can run two agent configurations side by side, collect metrics, and statistically determine winner. Test with model selection and prompt variations.
- **Dependencies:** P3-T13 (quality scoring)
- **Estimated Effort:** 1 day

- **Submodule(s) Involved:** HelixCode
- **Files/Modules to Modify:** tests/ab/*.go, pkg/ab/framework.go
- **Acceptance Criteria:** A/B test runs to completion; metrics collected; statistical significance computed; winner selected correctly.

P5-T10: Documentation and Runbook

- **Task ID:** P5-T10
- **Task Name:** Integration Documentation and Operational Runbook
- **Description:** Write comprehensive documentation: architecture diagrams, API docs, configuration reference, troubleshooting guide, and operational runbook for on-call engineers.
- **Dependencies:** All previous tasks
- **Estimated Effort:** 2 days
- **Submodule(s) Involved:** All
- **Files/Modules to Modify:** docs/INTEGRATION.md, docs/ARCHITECTURE.md, docs/OPERATIONS.md, docs/API.md
- **Acceptance Criteria:** All docs reviewed and approved; architecture diagrams current; runbook covers 10 common incidents; API docs match OpenAPI spec.

Section 4: SUBMODULE INTEGRATION SPECIFICATIONS

Submodule 1: HelixAgent

Attribute	Specification
Repository	git@github.com:HelixDevelopment/HelixAgent.git
Target Path	submodules/HelixAgent
Purpose	Parent/ancestor repository containing 60+ cli_agents integrations and 26+ submodules. Provides reference implementations, agent role definitions, and CLI agent patterns.
Special Consideration	HelixCode is itself a submodule OF HelixAgent. Adding HelixAgent as a submodule of HelixCode creates a circular dependency. Must be handled carefully.

Exact SSH Commands

```
# From HelixCode root
cd /path/to/HelixCode

# Add HelixAgent as submodule (shallow clone, specific branch)
git submodule add --name HelixAgent \
  git@github.com:HelixDevelopment/HelixAgent.git \
  submodules/HelixAgent

# Configure to not recurse into HelixAgent's submodules (avoid infinite recursion)
git config submodule.HelixAgent.update none
cd submodules/HelixAgent
```

```
git submodule deinit -f . # Prevent recursive init

# Fetch to known good commit (DO NOT checkout HelixCode's commit within HelixAgent)
git fetch origin
git checkout <TARGET_COMMIT> # e.g., v2.3.1 or specific SHA

# Return to HelixCode root and commit .gitmodules
cd ../../
git add .gitmodules submodules/HelixAgent
git commit -m "feat: add HelixAgent submodule for cli_agent integration reference"
```

Integration Wiring Steps

- 1. **Read-Only Reference:** HelixAgent is a READ-ONLY reference. Do NOT modify it from Helix-Code.
- 2. **Code Port Pattern:** When porting a feature from HelixAgent’s cli_agents:
 - Copy the relevant code into HelixCode’s pkg/ tree
 - Adapt import paths from dev.helix.agent to dev.helix.code
 - Add attribution comments referencing original source
 - Write tests for the ported code
- 3. **Interface Alignment:** HelixAgent’s pkg/agent/interfaces.go should be compared with Helix-Code’s actor model interfaces. Create adapter layer in pkg/agent/adapter.go if needed.
- 4. **No Build Dependency:** HelixCode must build without HelixAgent submodule being present. Use build tags: //go:build helixagent_ref for code that imports from HelixAgent.

Configuration

```
# config/helixagent.yaml (new)
helixagent:
  enabled: true
  path: "submodules/HelixAgent"
  reference_only: true
  cli_agents_path: "submodules/HelixAgent/cli_agents"
  integrations_path: "submodules/HelixAgent/integrations"
```

Build System Changes

- Add submodules/HelixAgent to go.work but NOT as a use directive (it has its own go.work)
- Instead, reference specific packages via replace if absolutely necessary
- Prefer copying code over cross-module imports to avoid circular build issues

Submodule 2: HelixLLM

Attribute	Specification
Repository	git@github.com:HelixDevelopment/HelixLLM.git
Target Path	submodules/HelixLLM

Table 9 – continued

Attribute	Specification
Purpose	Dedicated LLM provider management, model routing, prompt engineering, and token counting. Will be the PRIMARY provider of LLM capabilities, eventually replacing HelixCode's internal factory.

Exact SSH Commands

```
cd /path/to/HelixCode

# Add HelixLLM submodule
git submodule add --name HelixLLM \
  git@github.com:HelixDevelopment/HelixLLM.git \
  submodules/HelixLLM

cd submodules/HelixLLM
git fetch origin
git checkout <TARGET_COMMIT> # e.g., v1.5.0 or specific SHA

# Initialize and update its submodules (if any)
git submodule update --init --recursive

cd ../../
git add .gitmodules submodules/HelixLLM
git commit -m "feat: add HelixLLM submodule for unified LLM provider management"
```

Integration Wiring Steps

1. **Workspace Integration:** Add submodules/HelixLLM to go.work:

```
use ./submodules/HelixLLM
```

2. **Interface Migration:** Gradually replace pkg/llm/factory.go references with HelixLLM's provider registry:
 - Phase 1: Add adapter pkg/llm/helixllm_adapter.go
 - Phase 2: Port provider configurations
 - Phase 3: Migrate all provider calls (Phase 1 of this plan)
3. **Configuration Merge:** HelixLLM's provider config should be mergeable with HelixCode's Viper config tree. Use viper.MergeConfig() at initialization.
4. **API Exposure:** Expose HelixLLM's capabilities through HelixCode's Gin server via adapter handlers.

Configuration

```
# config/llm.yaml (new, merged into viper)
llm:
  gateway:
```

```

provider: "helixllm"
timeout: 60s
retry_policy: exponential
helixllm:
  module_path: "submodules/HelixLLM"
  providers:
    - openai
    - anthropic
    - google
    - mistral
    # ... 75+ providers

```

Build System Changes

- go.work: Add use ./submodules/HelixLLM
- go.work.sum: Will be auto-generated
- CI: Build HelixLLM independently before HelixCode to catch upstream issues early

Submodule 3: HelixMemory

Attribute	Specification
Repository	git@github.com:HelixDevelopment/HelixMemory.git
Target Path	submodules/HelixMemory
Purpose	Persistent conversation memory, embedding storage, semantic search, and cross-session context retrieval. Uses PostgreSQL + pgvector.

Exact SSH Commands

```

cd /path/to/HelixCode

git submodule add --name HelixMemory \
  git@github.com:HelixDevelopment/HelixMemory.git \
  submodules/HelixMemory

cd submodules/HelixMemory
git fetch origin
git checkout <TARGET_COMMIT>
git submodule update --init --recursive

cd ../../
git add .gitmodules submodules/HelixMemory
git commit -m "feat: add HelixMemory submodule for persistent conversation memory"

```

Integration Wiring Steps

1. **Database Schema:** HelixMemory manages its own migrations. HelixCode must:
 - Run HelixMemory migrations BEFORE starting the server

- Use migration table prefix `helixmemory_` to avoid collision with `HelixCode` tables
2. **Connection Sharing:** `HelixMemory` and `HelixCode` share the same PostgreSQL instance but separate schemas if needed. Use `search_path` or explicit schema qualification.
 3. **API Layer:** `HelixCode` wraps `HelixMemory`'s Go API in HTTP handlers under `/api/v1/memory/*`
 4. **Redis Integration:** `HelixMemory`'s cache layer should use `HelixCode`'s Redis connection pool (configured in Phase 1).

Configuration

```
# config/memory.yaml (merged into viper)
memory:
  enabled: true
  module_path: "submodules/HelixMemory"
  database:
    schema: "helixmemory"
    migration_path: "submodules/HelixMemory/migrations"
  embedding:
    model: "text-embedding-3-large"
    dimensions: 3072
  retrieval:
    top_k: 5
    similarity_threshold: 0.85
```

Build System Changes

- go.work: Add use `./submodules/HelixMemory`
- PostgreSQL migration runner: Update to include `submodules/HelixMemory/migrations/*.sql`
- Docker Compose: Add pgvector-enabled PostgreSQL image

Submodule 4: HelixSpecifier

Attribute	Specification
Repository	git@github.com:HelixDevelopment/HelixSpecifier.git
Target Path	submodules/HelixSpecifier
Purpose	Task decomposition, specification generation, and structured output validation. Powers Plan Mode and the architect/editor workflow.

Exact SSH Commands

```
cd /path/to/HelixCode

git submodule add --name HelixSpecifier \
  git@github.com:HelixDevelopment/HelixSpecifier.git \
  submodules/HelixSpecifier

cd submodules/HelixSpecifier
git fetch origin
```

```
git checkout <TARGET_COMMIT>
git submodule update --init --recursive

cd ../../
git add .gitmodules submodules/HelixSpecifier
git commit -m "feat: add HelixSpecifier submodule for task decomposition and planning"
```

Integration Wiring Steps

1. **LLM Dependency:** HelixSpecifier needs LLM access. Route through HelixCode's LLM gateway (Phase 1), NOT directly to providers.
2. **Structured Output:** HelixSpecifier's JSON schema validation should use HelixCode's validation library (or shared library).
3. **Spec Storage:** Generated specifications stored in HelixMemory for retrieval and version tracking.
4. **Plan Mode Integration:** HelixSpecifier is the engine behind Plan Mode (Phase 3, P3-T1). Direct call from pkg/plan/engine.go to HelixSpecifier.

Configuration

```
# config/specifier.yaml (merged into viper)
specifier:
  enabled: true
  module_path: "submodules/HelixSpecifier"
  llm:
    provider: "helixllm"
    model: "claude-3-5-sonnet"
    temperature: 0.2
  decomposition:
    max_depth: 3
    min_confidence: 0.8
  validation:
    schema_strict: true
    retry_on_failure: 3
```

Build System Changes

- go.work: Add use ./submodules/HelixSpecifier
- If HelixSpecifier has its own LLM client, replace with adapter to HelixCode gateway

Summary of Submodule Additions

```
# One-shot script for all 4 submodules (run from HelixCode root)
#!/bin/bash
set -e

REPOS=(
  "git@github.com:HelixDevelopment/HelixAgent.git:submodules/HelixAgent"
  "git@github.com:HelixDevelopment/HelixLLM.git:submodules/HelixLLM"
  "git@github.com:HelixDevelopment/HelixMemory.git:submodules/HelixMemory"
```

```

"git@github.com:HelixDevelopment/HelixSpecifier.git:submodules/HelixSpecifier"
)

for repo_spec in "${REPOS[@]"; do
    IFS=: read -r url path <<< "$repo_spec"
    name=$(basename "$path")

    echo "=== Adding $name ==="
    git submodule add --name "$name" "$url" "$path"

    cd "$path"
    git fetch origin
    # NOTE: Replace <TARGET_COMMIT> with actual commit after audit
    git checkout <TARGET_COMMIT>
    git submodule update --init --recursive || true
    cd ../../

done

# Update go.work
cat >> go.work <<EOF
use ./submodules/HelixLLM
use ./submodules/HelixMemory
use ./submodules/HelixSpecifier
# HelixAgent NOT added to go.work (circular dependency, read-only reference)
EOF

git add .gitmodules go.work
git commit -m "feat: add HelixLLM, HelixMemory, HelixSpecifier submodules; HelixAgent as reference"

```

Section 5: DEPENDENCY GRAPH

5.1 Visual Representation

```

PHASE 0: FOUNDATION
=====
[P0-T1] Audit Submodules
    |
    v
[P0-T2] Add Missing Submodules --+-----+
    |                               |
    v                               |
[P0-T3] Initialize Dormant        |
    |                               |
    v                               |
[P0-T4] Create Go Workspace -----+
    |                               |
    v                               |
[P0-T5] CI Pipeline -----+
    |

```



```

      v
[P3-T5] Container Sandbox ---+
      |
      v
[P3-T6] Native Sandbox
      |
      v
[P3-T8] MCP Client <-----+
      |
      v
[P3-T9] Hook System <-----+
      |
      v
[P3-T10] Skill System
      |
      v
[P3-T13] Quality Scoring
      |
      v
[P3-T14] OpenTelemetry

      v
[P3-T11] Subagent Team
      |
      v
[P3-T12] Browser
      |
      v
[P3-T7] Permissions
      |
      v

PHASE 4: UI/UX & TUI
=====
[P4-T1] Streaming TUI <-----+
      |
      v
[P4-T2] Theme System
      |
      v
[P4-T3] AskUserQuestion Previews
      |
      v
[P4-T6] Dual-Model UI
      |
      v
[P4-T7] State Machine UI
      |
      v
[P4-T9] Multimodal Input

      |
      v
[P4-T4] Terminal Intellisense
      |
      v
[P4-T5] Slash Commands
      |
      v
[P4-T8] LSP Integration
      |
      v
[P4-T10] 1M Context UI

PHASE 5: TESTING & QA
=====
[P5-T1] 100% Coverage
      |
      v
[P5-T2] Integration Tests
      |
      v
[P5-T3] HelixQA Challenges
      |
      v

```

```
[P5-T4] LLMsVerifier
|
v
[P5-T5] Security Audit
|
v
[P5-T6] Performance Benchmarks
|
v
[P5-T7] Load Testing
|
v
[P5-T8] Plugin Validation
|
v
[P5-T9] A/B Testing
|
v
[P5-T10] Documentation
```

5.2 Critical Path Identification

The critical path is the longest dependency chain that determines minimum project duration:

```
CRITICAL PATH (approximate duration: 76-95 days)
=====
P0-T1 (1d) -> P0-T2 (1d) -> P0-T4 (2d) -> P0-T5 (2d)
    [Phase 0 total: 6d]
    |
    v
P1-T1 (2d) -> P1-T2 (4d) -> P1-T6 (3d)
    [Phase 1 critical: 9d]
    |
    v
P2-T1 (3d) -> P2-T2 (3d) -> P2-T6 (3d) -> P2-T7 (2d)
    [Phase 2 critical: 11d]
    |
    v
P3-T1 (3d) -> P3-T5 (4d) -> P3-T6 (3d) -> P3-T8 (5d)
    [Phase 3 critical: 15d]
    |
    v
P4-T1 (3d) -> P4-T2 (2d) -> P4-T3 (2d)
    [Phase 4 critical: 7d]
    |
    v
P5-T1 (5d) -> P5-T2 (3d) -> P5-T3 (3d) -> P5-T6 (2d) -> P5-T7 (2d)
    [Phase 5 critical: 15d]
```

Critical Path Total: ~63 working days (~76-95 calendar days with weekends/holidays/buffer)

5.3 Parallel Workstreams

Multiple workstreams can proceed in parallel once Phase 0 is complete:

Workstream A: LLM & Providers (Phase 1 focus)

- P1-T1 -> P1-T2 -> P1-T3 -> P1-T8
- Can start immediately after P0-T5

Workstream B: Memory & Retrieval (Phase 1 focus)

- P1-T4 -> P1-T5 -> P1-T7
- Can start immediately after P0-T5

Workstream C: Specification Engine (Phase 1 focus)

- P1-T1 -> P1-T6 -> (feeds into Phase 3 Plan Mode)
- Can start immediately after P1-T1

Workstream D: Tool Framework (Phase 2 focus)

- P2-T1 -> P2-T2 -> P2-T3
- Needs P1-T2 (for LLM tool)

Workstream E: Context Management (Phase 2 focus)

- P2-T4 -> P2-T5 -> P2-T8
- Needs P1-T5 (embeddings) and P1-T7 (Redis)

Workstream F: UI/UX Foundation (Phase 4 focus)

- P4-T1 -> P4-T2 -> P4-T3
- Needs P2-T7 (no-flicker rendering)
- Can start mid-Phase 2 once P2-T7 is complete

Workstream G: Sandboxing (Phase 3 focus)

- P3-T5 -> P3-T6
- Needs P0-T3 (Containers submodule)

Workstream H: MCP Integration (Phase 3 focus)

- P3-T8
- Needs P1-T2 and P2-T1

Workstream I: Quality & Testing (Phase 5 focus)

- P5-T5 -> P5-T6 -> P5-T7
- Needs P3-T5 and P3-T6 complete

5.4 Merge Points (High Coordination Required)

Merge Point	Tasks Converging	Coordination Need
M1 (After P2-T2)	Tool suite complete	All Phase 2+ tool-dependent tasks can start
M2 (After P3-T1)	Plan mode ready	UI, permissions, MCP can integrate with plan mode
M3 (After P3-T5)	Sandboxing ready	Security audit, integration tests can begin
M4 (After P4-T1)	TUI streaming ready	All UI features can integrate
M5 (After P5-T2)	Integration tests pass	Final QA and release preparation

Section 6: RISK REGISTER

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-01	Circular submodule dependency causes build failures: HelixCode is a child of HelixAgent. Adding HelixAgent as a submodule of HelixCode creates a loop that git may refuse to recurse.	4	5	20	Add HelixAgent as shallow, non-recursive submodule with <code>update=none</code> config. Do NOT add to <code>go.work</code> . Use read-only reference pattern.	If git rejects the submodule add, maintain HelixAgent as a completely separate clone (e.g., <code>../HelixAgent</code>) and reference via symlink or documentation. Do not block build on HelixAgent presence.
R-02	SSH key authentication fails in CI/CD: GitHub Actions (or other CI) cannot access private repos via SSH without deployed keys.	4	4	16	Configure Deploy Keys or GitHub App tokens for CI. Use <code>webfactory/ssh-agent</code> action. Document key rotation process.	Fallback to HTTPS with Personal Access Tokens for CI. Rotate tokens via GitHub Secrets. Document token scoping (read-only).

Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-03	Go module version conflicts across 11 submodules: Different submodules may depend on different versions of shared libraries (e.g., Gin, Cobra, Viper, Redis client).	4	4	16	Centralize dependency version management in <code>go.work</code> . Run <code>go work sync</code> regularly. Pin major versions in workspace.	Use <code>replace</code> directives in <code>go.work</code> temporarily to force version alignment. Create dependency upgrade sprint after integration.
R-04	LLM provider API drift breaks gateway: Provider APIs change (OpenAI, Anthropic, Google), causing runtime failures after integration.	4	3	12	Implement provider capability discovery and version negotiation. Abstract provider specifics behind gateway interface. Add health checks with automatic failover.	Maintain provider-specific fallback adapters. Use multiple providers for critical paths (automatic failover). Alert on provider health check failures.
R-05	Context compaction loses critical semantic information: Auto-compaction may remove tokens that are essential for task completion, causing agent failure.	3	4	12	Multi-layer compaction strategy: first summarize, then compress, then evict by relevance. Use embedding-based relevance scoring. Preserve tool results and user directives.	Implement “pin” mechanism for critical context. Allow user to mark context as non-compactable. Fallback to manual context management mode.

Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-06	Sandbox escape compromises host system: Container or seccomp-bpf misconfiguration allows malicious code to escape sandbox.	3	5	15	Defense in depth: containers + seccomp + AppArmor/SELinux + non-root user + read-only rootfs + limited capabilities. Regular security audits.	Immediately disable sandbox execution and fall back to read-only mode. Quarantine affected sessions. Engage security team for incident response.
R-07	Permission system locks out legitimate operations: Overly restrictive default permissions or wildcard misconfiguration prevents normal agent operation.	3	4	12	Default all permissions to ask (never deny by default). Require explicit opt-in for deny rules. Validate wildcard patterns before saving. Provide --emergency-override flag.	Emergency override flag bypasses all permission checks. Log all overrides for audit. Support gradual permission tightening after initial permissive period.
R-08	100% test coverage mandate causes schedule slip: Writing comprehensive tests for 20+ feature domains may take longer than estimated, pushing timeline.	4	3	12	Start test writing concurrently with feature development (TDD). Use code generation for boilerplate tests. Focus on critical path coverage first.	Allow 95% coverage as temporary gate with technical debt ticket for remaining 5%. Prioritize integration tests over unit tests for non-critical paths.

Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-09	Theme system has platform-specific rendering bugs: Terminal capability detection fails on exotic terminals, causing garbled output or crashes.	3	2	6	Extensive terminal capability matrix testing. Graceful degradation to dumb terminal mode. Platform detection with fallback chains.	UI_MODE=simpl forces plain text output. Per-platform test suites in CI. Community bug reporting for unsupported terminals.
R-10	MCP OAuth implementation complexity: OAuth 2.1 PKCE flow is complex, error-prone, and may not work with all MCP servers.	3	3	9	Use mature OAuth library (e.g., golang.org/x/oauth2). Implement comprehensive error handling. Test against reference MCP servers.	Support token-based auth as fallback for servers without OAuth. Document manual token configuration. Phase MCP OAuth as beta feature.
R-11	HelixQA challenge flakiness causes false negatives: Challenge tests may fail due to non-deterministic LLM output, not actual regressions.	4	3	12	Implement challenge retry with exponential backoff. Use deterministic seed where possible. Grade on multiple criteria (not just exact match). Maintain challenge stability score.	Quarantine flaky challenge after 3 consecutive failures. Manual review of borderline results. Separate “stable” and “experimental” challenge suites.

Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-12	Performance regression under load: 100 concurrent sessions may expose memory leaks, goroutine leaks, or database connection pool exhaustion.	3	4	12	Load test early (not just at end). Use pprof profiling. Set resource limits and alerts. Implement circuit breakers for LLM calls.	Horizontal scaling: run multiple HelixCode instances behind load balancer. Scale PostgreSQL read replicas. Implement request queuing and backpressure.
R-13	Key contributor availability: Core team members may be unavailable due to vacation, illness, or other projects.	3	3	9	Pair programming and knowledge sharing sessions. Document architecture decisions. Cross-train team members on critical subsystems.	Redistribute tasks based on availability. Use external contractors for well-defined, isolated tasks. Extend timeline if needed.
R-14	Tree-sitter integration gaps: Context compaction relies on Tree-sitter for structural summaries. Some languages may lack parser support.	2	3	6	Maintain fallback to regex-based heuristics for unsupported languages. Prioritize top-10 language parsers. Use LLM-based summarization as ultimate fallback.	Document supported languages. Graceful degradation to text-level compaction. Community contribution path for new Tree-sitter parsers.

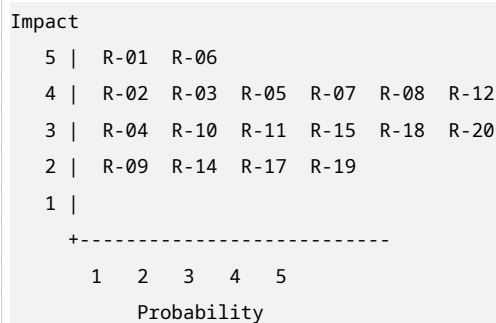
Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-15	Database migration conflicts: HelixMemory and HelixCode migrations may conflict on schema names, table names, or migration versioning.	3	3	9	Use separate PostgreSQL schemas. Centralized migration runner with ordering. Versioned migration filenames with timestamps. Pre-deployment migration test in staging.	Rename conflicting tables/columns with migration. Use schema separation (<code>helixmemory.*</code> vs <code>helixcode.*</code>). Emergency rollback via down migrations.
R-16	Skill system YAML parsing vulnerabilities: Loading skills from YAML files could introduce deserialization vulnerabilities if untrusted skills are loaded.	2	4	8	Strict YAML schema validation. Whitelist allowed keys. Sanitize all string inputs. Load skills from trusted paths only.	Disable skill loading via <code>--no-skills</code> flag. Review all skill files before loading. Use static analysis on skill definitions.
R-17	Embedding model compatibility: Embedding dimensions or models may change, invalidating stored vectors in pgvector.	2	3	6	Store embedding model version with vectors. Detect model mismatch on retrieval. Support re-embedding on model change.	Migration to re-embed all stored content. Dimension-agnostic storage if possible. Alert on model version mismatch.

Table 13 – continued

Risk ID	Risk Description	Probability (1-5)	Impact (1-5)	Risk Score (PxI)	Mitigation Strategy	Contingency Plan
R-18	Subagent communication deadlock: Multiple subagents waiting on each other creates deadlock in orchestrator.	2	4	8	Implement timeouts on all inter-agent requests. Use directed acyclic graphs for task dependencies. Monitor for circular wait conditions.	Timeout-based cancellation of deadlocked tasks. Fallback to single-agent execution. Log deadlock patterns for post-hoc analysis.
R-19	OpenTelemetry2 overhead impacts performance: Tracing and metrics collection adds latency to fast paths.	2	2	4	Use sampling (1% default, 100% on demand). Async trace export. Lightweight span creation. Benchmark with/without telemetry.	Disable telemetry via OTEL_DISABLED. Reduce sampling rate. Batch span export to reduce overhead.
R-20	Mid-session provider switching loses context: Switching LLM providers mid-session may cause context format incompatibilities.	3	3	9	Normalize context format in HelixCode before sending to any provider. Abstract provider-specific message formats. Test context round-trip for each provider.	Warn user before switching. Offer to compact/summarize context on switch. Maintain per-provider context caches.

Risk Heat Map



HIGH RISK ZONE (PxI >= 12): R-01, R-02, R-03, R-04, R-05, R-06, R-07, R-08, R-11, R-12, R-20

MEDIUM RISK ZONE (PxI 6-11): R-09, R-10, R-13, R-14, R-15, R-16, R-17, R-18

LOW RISK ZONE (PxI < 6): R-19

APPENDIX A: Feature Inventory Summary

Source Agent	# Features	Key Features Integrated
Claude Code	20	Auto-Compaction, Permissions, Tool Persistence, Worktree Isolation, Hooks, No-Flicker Rendering, Background Tasks, Smart Editing, Plan Mode, Slash Commands, MCP Lifecycle, Skills, Session Resume, Multi-Provider Wizards, LSP, Sandboxed Shell, Themes, AskUserQuestion, Subagent Team, OpenTelemetry
Aider	3	Architect/Editor Dual-Model, 4-Layer Fuzzy Matching, Git-Native Auto-Commit
Cline	3	Plan/Act with Different Models, Shadow Git Checkpoints, Browser Automation
Codex	3	OS-Native Sandboxed Execution, Automatic Context Compaction, Stateless ZDR Architecture
Plandex	3	Cumulative Diff Review Sandbox, 2M Token Context + 20M Indexing, Version-Controlled Model Packs
Forge	3	6 Orchestration Patterns, Quality Scoring with Gates, A/B Testing for Agent Configs
Kilo Code	3	5 Specialized Modes + Subagent Delegation, Multi-Agent Platform State Machine, Auto Triage/Review/App Builder
OpenCode	3	75+ Provider Support with Mid-Session Switching, LSP Integration with Diagnostics, Plugin System with 25+ Hooks
Gemini CLI	3	1M Token Context Window, Plan Mode with ask_user, Multimodal Input
Amazon Q	3	Fig-Style Terminal Intellisense, Architecture Diagram <-> Code Sync, CDK Construct Abstraction

Total Feature Domains: 47 distinct features across 10 source agents

APPENDIX B: Timeline Summary

Phase	Calendar Days	Cumulative Days	Risk Level
Phase 0: Foundation	5-7	5-7	HIGH
Phase 1: Core Infrastructure	14-18	19-25	HIGH
Phase 2: CLI Agent Foundation	18-22	37-47	MEDIUM-HIGH
Phase 3: Power Features	21-28	58-75	HIGH
Phase 4: UI/UX & TUI	14-18	72-93	MEDIUM
Phase 5: Testing & QA	14-18	86-111	MEDIUM

Estimated Total Duration: 86-111 calendar days (~3-4 months with 1 FTE team of 4-6 engineers)
Go-Live Target: Q2 2025 (with buffer for risk mitigation)

APPENDIX C: Team Assignment Recommendations

Role	Responsibilities	Phases
Integration Lead	Architecture decisions, submodule coordination, critical path management	All
LLM Engineer	Provider gateway, multi-model support, context compaction	1, 2, 3
Agent Engineer	Tool framework, plan mode, subagent orchestration	2, 3
Infrastructure Engineer	Sandboxing, MCP, security, build system	0, 1, 3, 5
UI/UX Engineer	TUI streaming, themes, terminal intellisense, multimodal	4
QA Engineer	Test strategy, coverage enforcement, challenge design	5 (support throughout)
DevOps Engineer	CI/CD, deployment, observability, load testing	0, 1, 5

End of Master Integration Plan